Lecture No.30

Trees: basic definitions and terminology

Contrary to arrays, stacks, queues and sequences all of which are one-dimensional data structures, trees are **two-dimensional data structures with hierarchical relationship between data items.**

Definition 1 A **tree** is a non-empty collection of vertices (nodes) and edges that satisfy certain requirements.

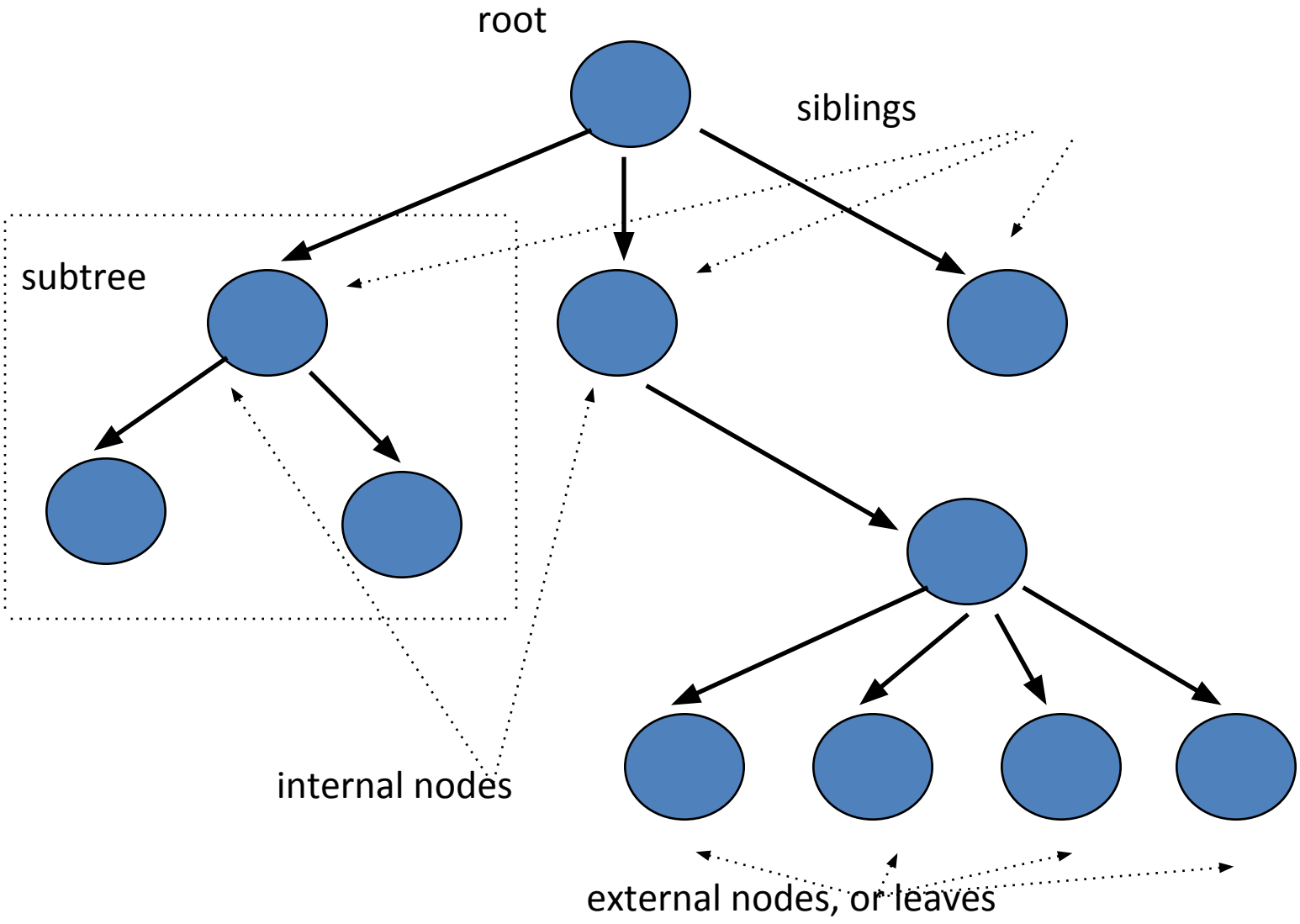
Definition 2 A **path** in a tree is a list of distinct vertices in which successive vertices are connected by edges in the tree.

One node in the tree is designated as the **root**. Each tree has exactly one path between the root and each of the other nodes. If there is more than one path between the root and some node, or no path at all, we have a **graph**.

Definition 3 A set of trees is called a **forest**.

Definition 4 (Recursive definition) A **tree** is either a single node or a root node connected to a forest.

Example of a tree:



More definitions

Definition 5 An **ordered tree** is a tree in which the order of children is specified.

Definition 6 A **level** (**depth**) of a node in the number of nodes on the path from that node to the root.

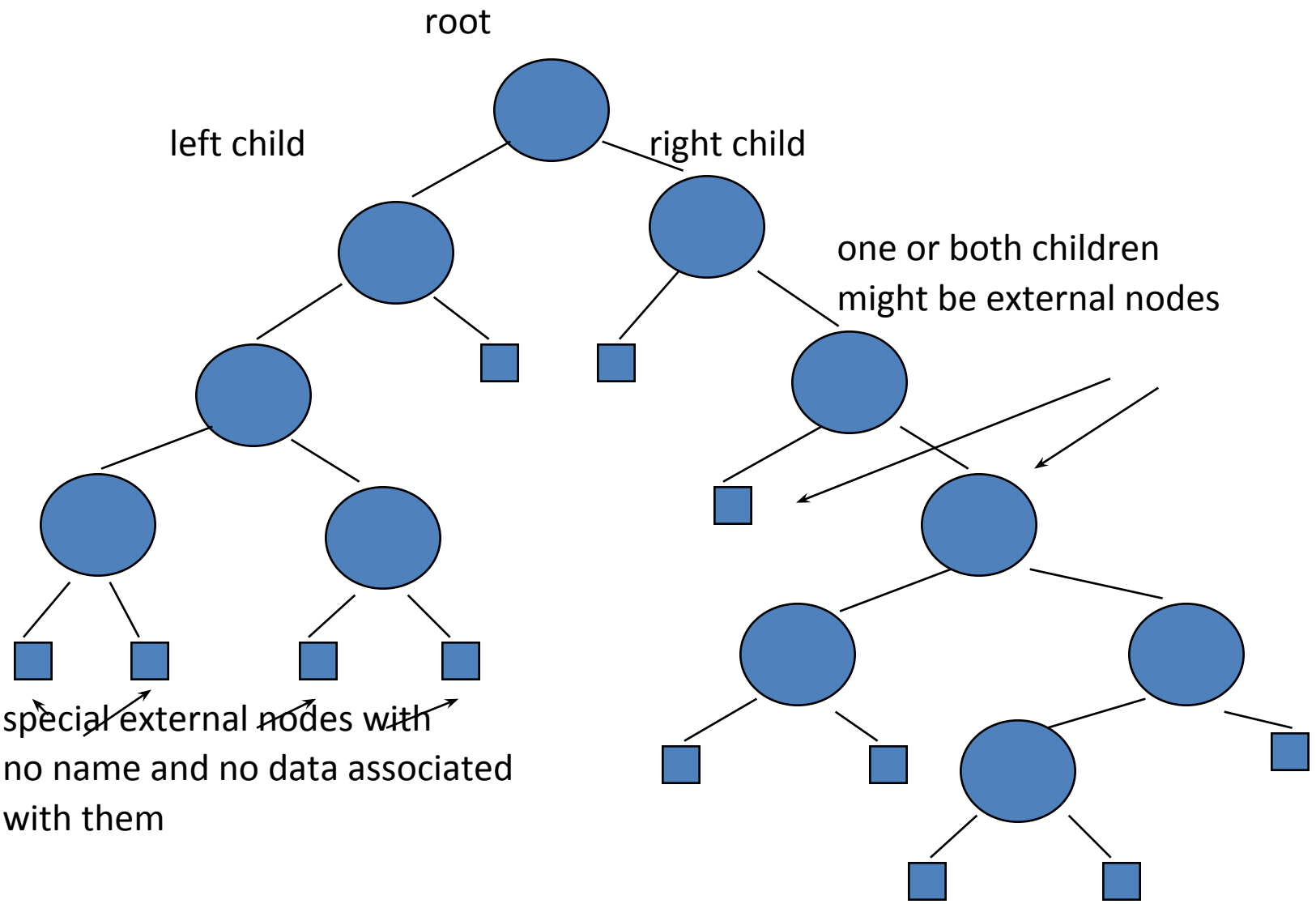
Definition 7 The **height** (**maximum distance**) of a tree is the maximum level among all of the nodes in the tree.

Definition 8 The **path length** of a tree is the sum of the levels of all the nodes in the tree.

Definition 9 A tree where each node has a specific number of children appearing in a specific order is call a **multiway tree**. The simplest type of a multiway tree is the **binary tree**. Each node in a binary tree has exactly two children one of which is designated as a left child, and the other is designated as a right child.

Definition 10 (Recursive definition) A binary tree is either an external node, or an internal node and two binary trees.

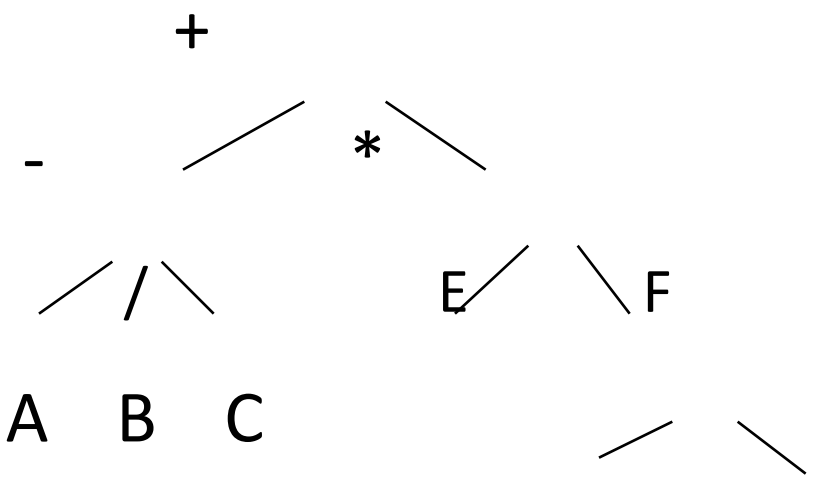
Example of a binary tree



More binary trees examples

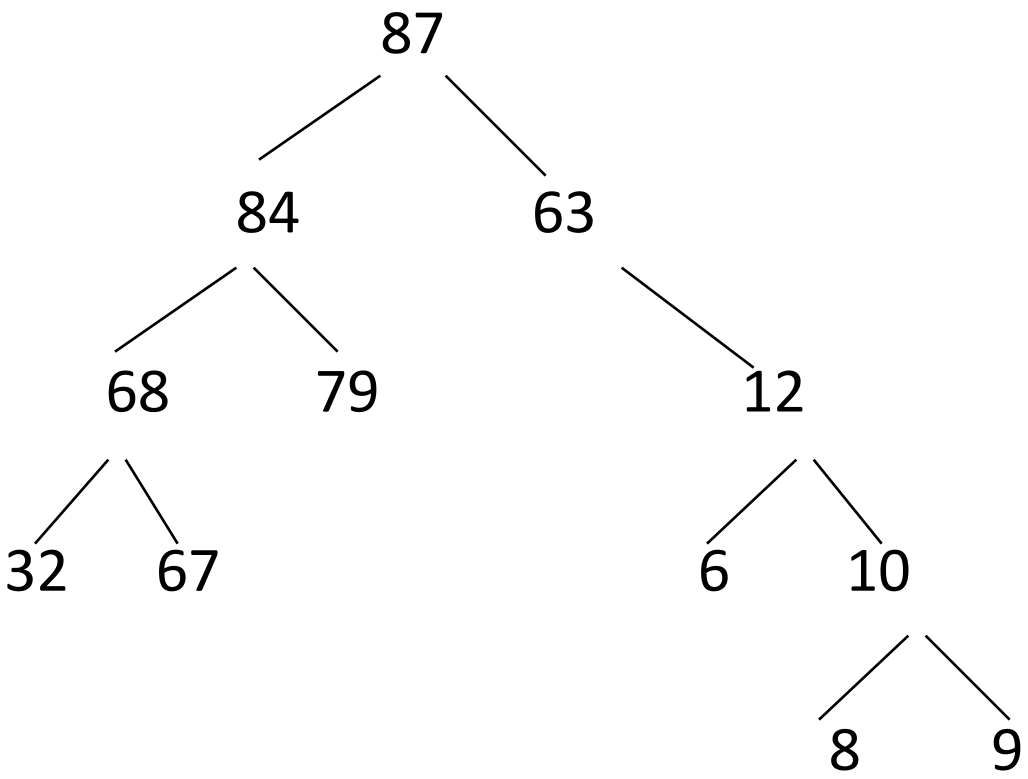
1. Binary tree for representing arithmetic expressions. The underlying hierarchical relationship is that of an arithmetic operator and its two operands.

   Arithmetic expression in an infix form: (A - B) + C * (E / F)

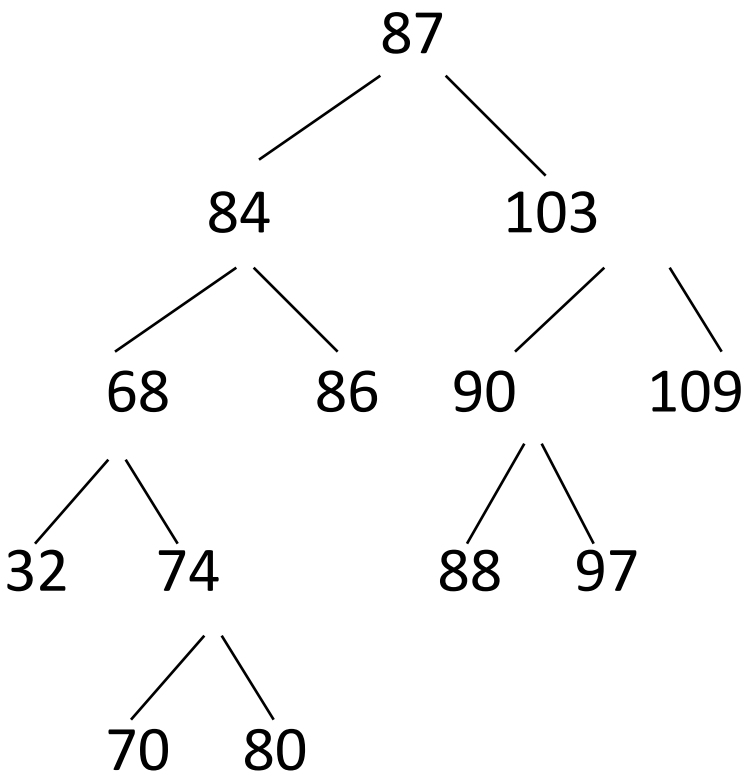


Note that a post-order traversal of this tree (i.e. visiting the left subtree first, right subtree next, and finally the root) returns the postfix form of the arithmetic expression, while the pre-order traversal (root is visited first, then the left subtree, then the right subtree) returns the prefix form of the arithmetic expression.

2. Binary tree with a heap property. The underlying hierarchical relationship suggests that the datum in each node is greater than or equal to the data in its left and right subtrees.

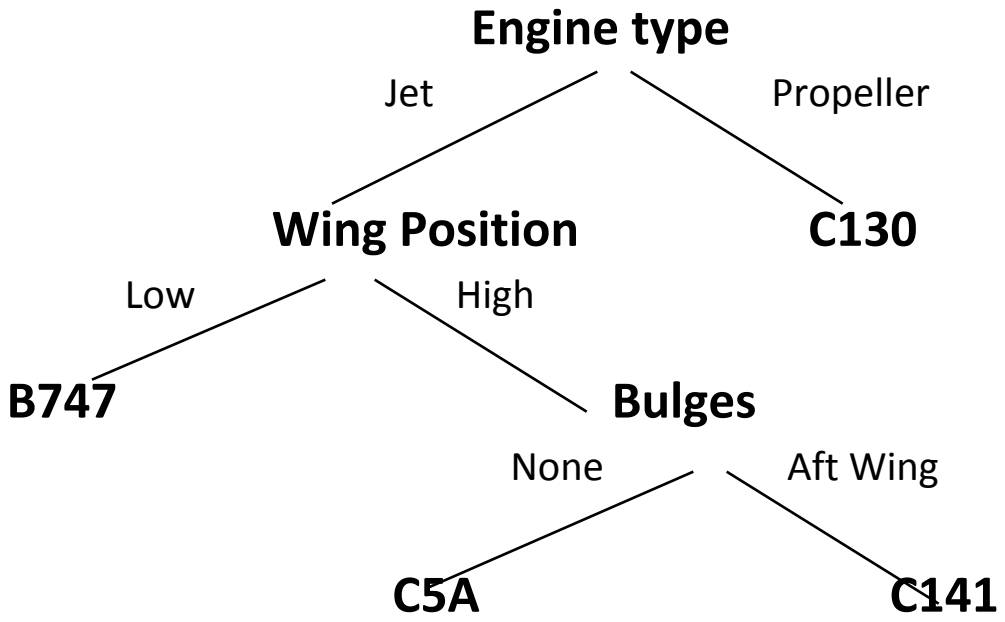


3. Binary tree with an ordering property. The underlying hierarchical relationship suggests that the datum in each node is greater than the data in its left subtree, and less than or equal to the data in its right subtrees.



4. Decision trees. The underlying hierarchical relationship depends on the nature of the domain represented by the binary tree. For example, consider a domain that consists of the following statements (from J.Ignizio “Intro to ES”):
   - If the plane’s engine is propeller, then the plane is C130.
   - If the plane’s engine is jet and the wing position is low, then the plane is B747.
   - If the plane’s engine is jet and the wing position is high and no bulges are seen, then the plane is C5A
   - If the plane’s engine is jet and the wing position is high and bulges are aft of wing, then plane is C141 .

   The following decision tree can be generated from these rules:



Properties of binary trees

1. The number of external nodes is 1 more than the number of internal nodes. It is easy to see this if we start removing external nodes with their internal parent, one pair at a time (assume that a method removeAboveExternal(n) does this). At the end of this process, only the root with its two external children will remain.

2. The number of external nodes is at least **h + 1**, where **h** is the height of the tree, and at most $\mathbf{2^h}$. The later holds for a **full binary tree**, which is a tree where internal nodes completely fill every level.

3. The number of internal nodes is at least **h** and at most $\mathbf{2^h - 1.}$

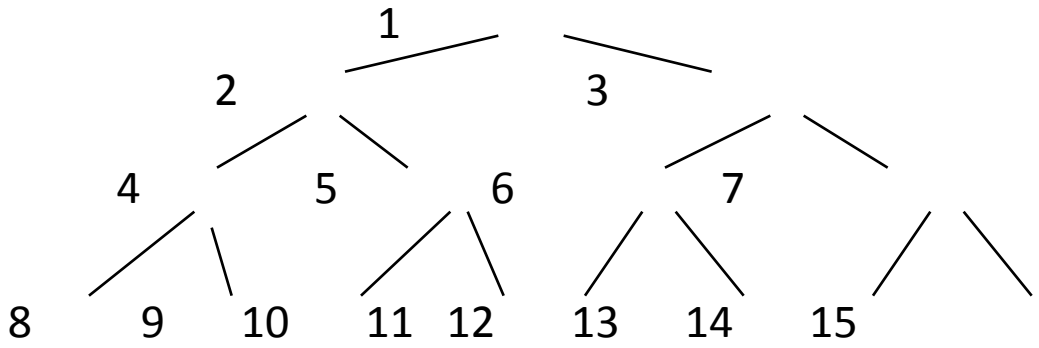
4. The total number of nodes in a binary tree is at least **2*h + 1** and at most $\mathbf{2^{h+1} - 1.}$

5. The height, **h**, of a binary tree with **n** nodes is at least **log n+1** and at most **n**.

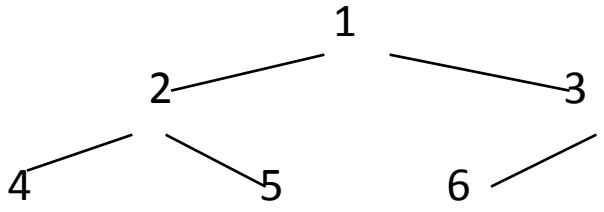
6. A binary tree with **n** nodes has exactly **n - 1** edges.

Full binary trees and complete binary trees

Here is an example of a full binary tree:



A complete binary tree is a full binary tree where the internal nodes on the bottom level all appear to the left of the external nodes on that level. Here is an example of a complete binary tree:



A **Binary Search Tree** is a binary tree with the following properties:

- All items in the left subtree are less than the root.
- All items in the right subtree are greater or equal to the root.
- Each subtree is itself a binary search tree.

Basic Property

- In a binary search tree,
- the left subtree contains key values **less** than the root
- the right subtree contains key values **greater** than or **equal** to the root.

Binary search trees provide an excellent structure for searching a list and at the same time for inserting and deleting data into the list.

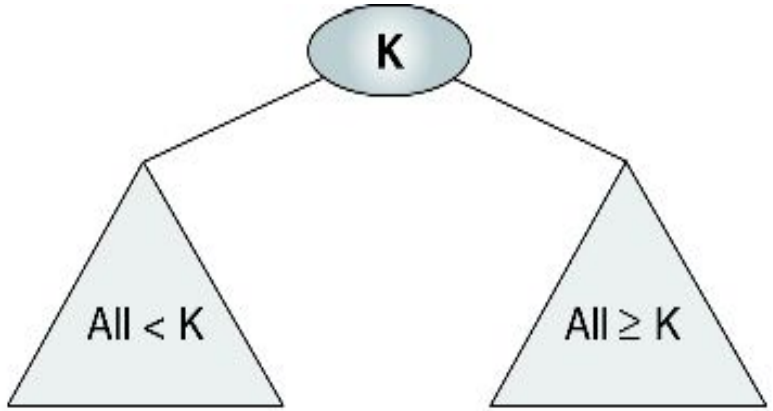


FIGURE 7-1 Binary Search Tree

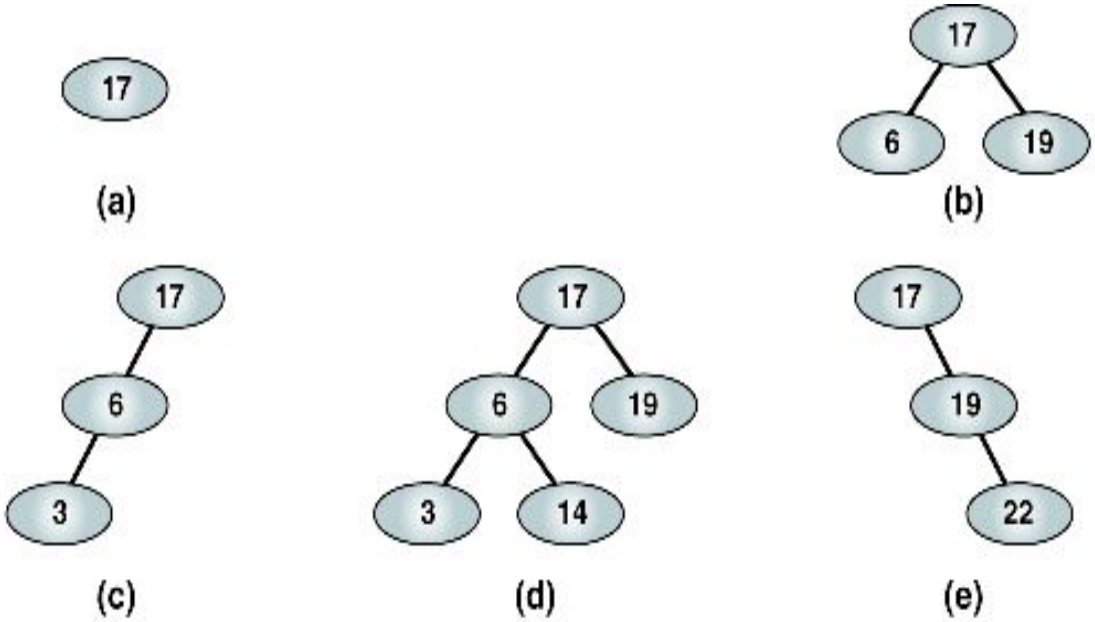


FIGURE 7-2 Valid Binary Search Trees

(a), (b) - complete and balanced trees;

(d) – nearly complete and balanced tree;

(c), (e) – neither complete nor balanced trees

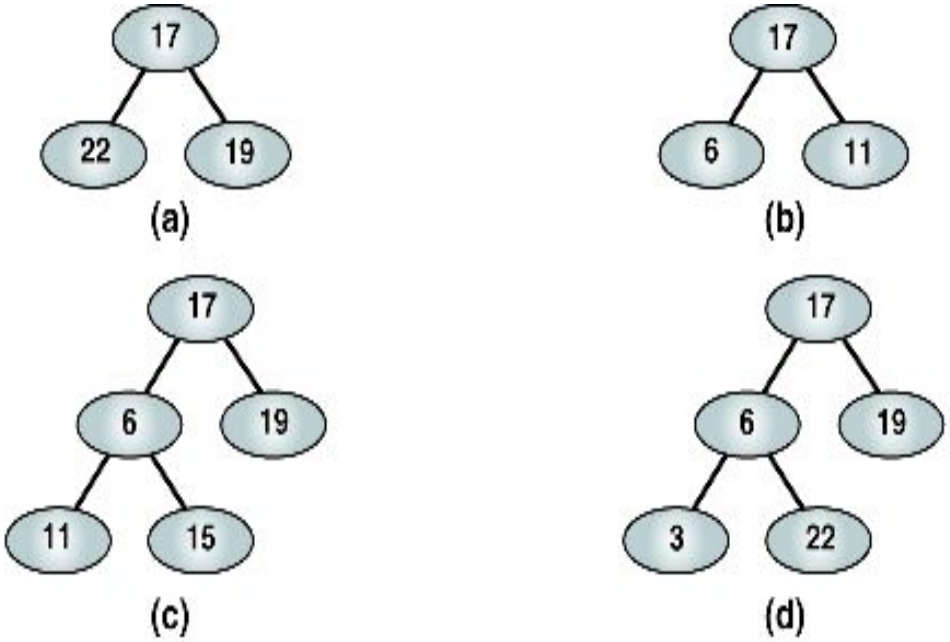


FIGURE 7-3 Invalid Binary Search Trees

7-2 BST Operations

We discuss four basic BST operations: traversal, search, insert, and delete; and develop algorithms for searches, insertion, and deletion.

- Traversals
- Searches
- Insertion
- Deletion

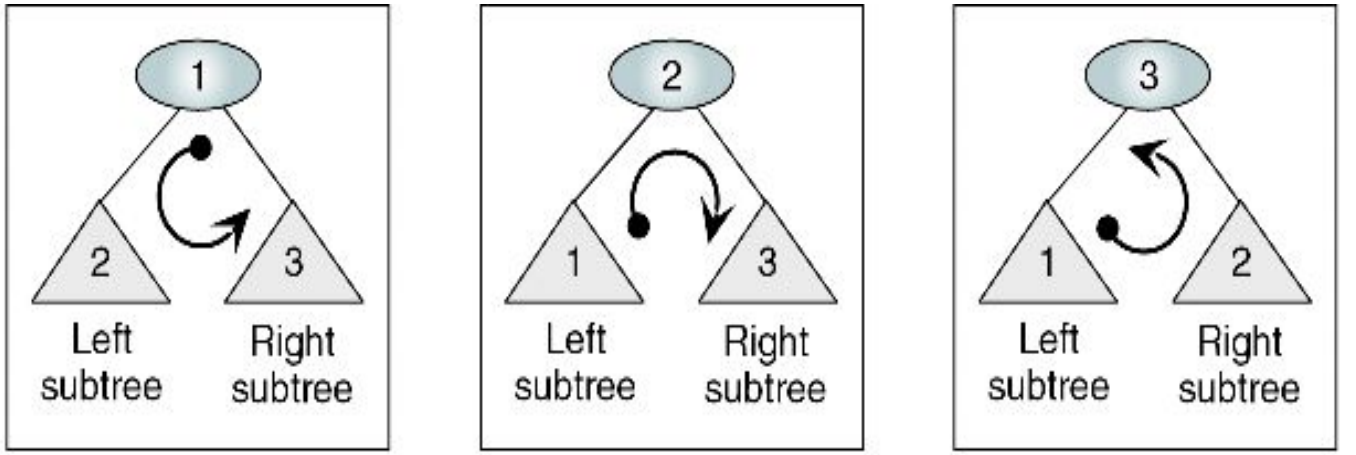


FIGURE 6-8 Binary Tree Traversals

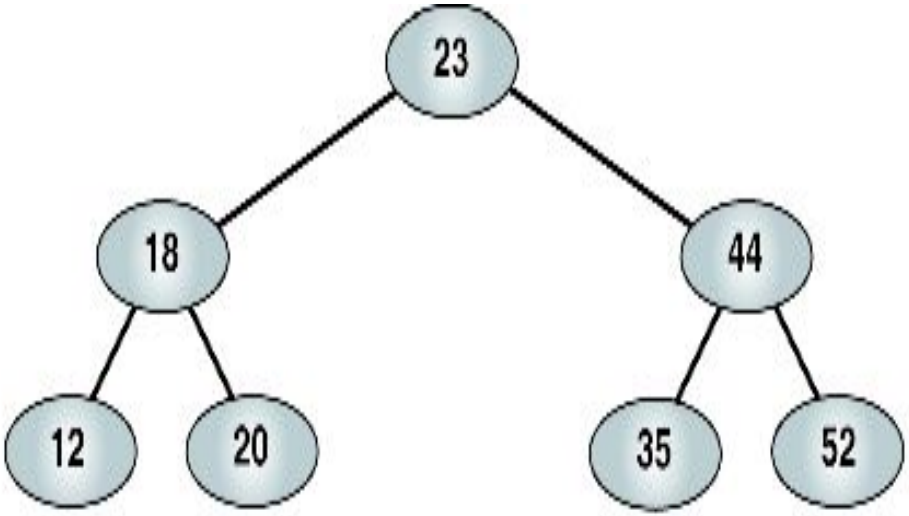


FIGURE 7-4 Example of a Binary Search Tree

Preorder Traversal

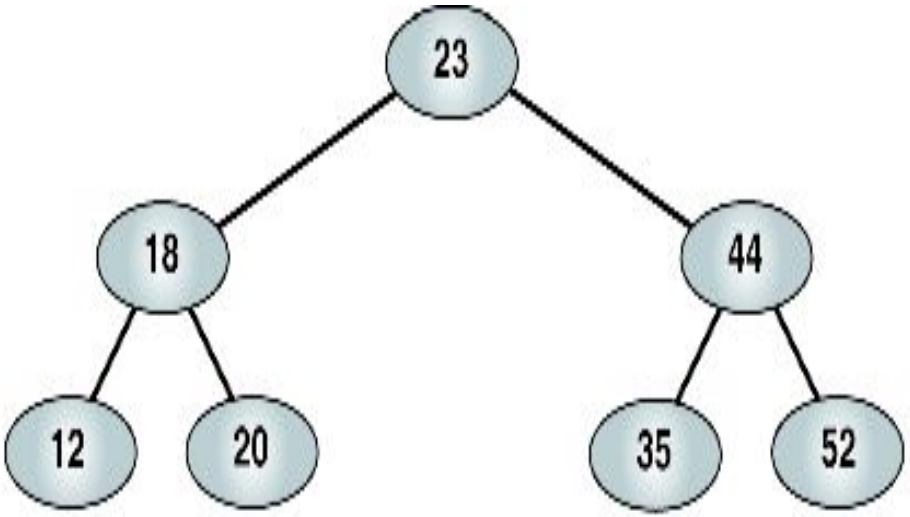


FIGURE 7-4 Example of a Binary Search Tree

23 18 12 20 44 35 52

Postorder Traversal

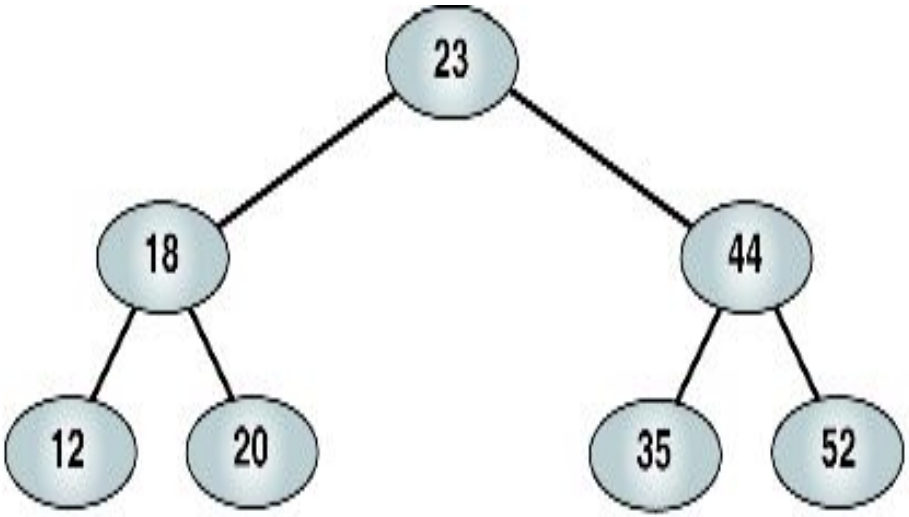


FIGURE 7-4 Example of a Binary Search Tree

12 20 18 35 52 44 23

Inorder Traversal

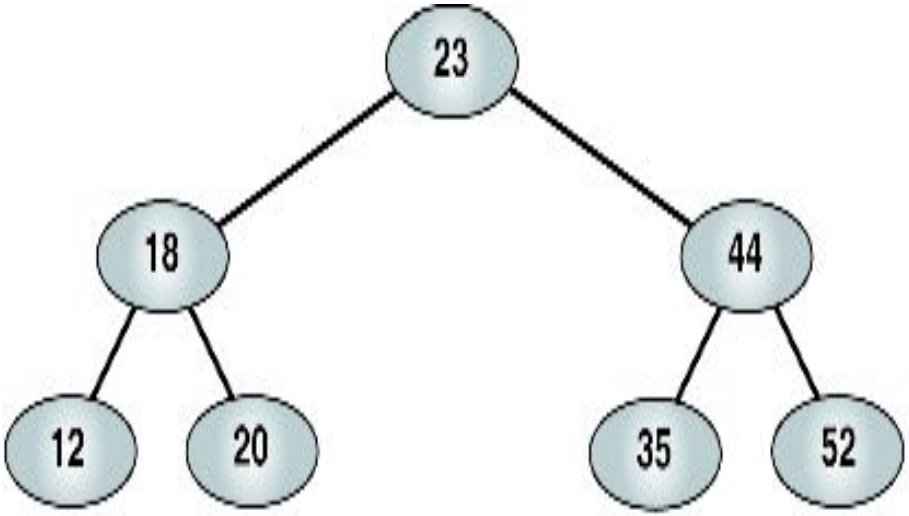


FIGURE 7-4 Example of a Binary Search Tree

12 18 20 23 35 44 52

Inorder traversal of a binary search tree produces a sequenced list

Right-Node-Left Traversal

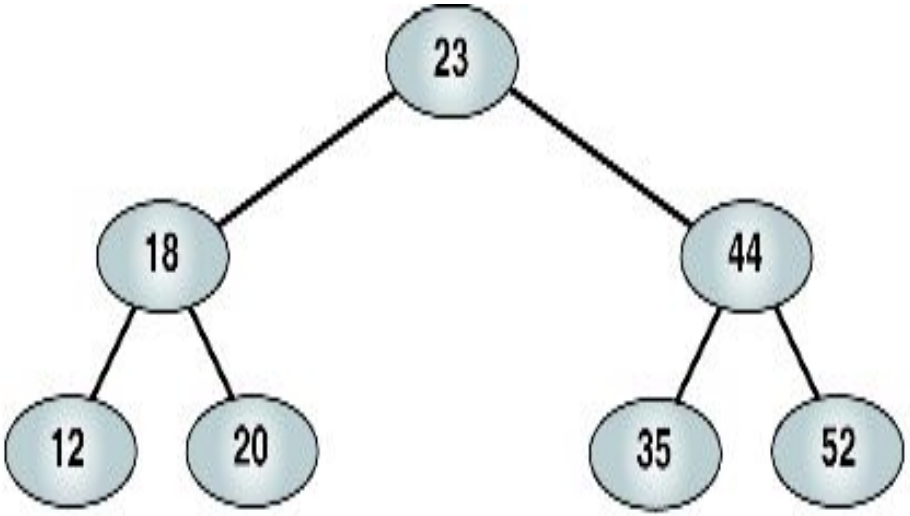


FIGURE 7-4 Example of a Binary Search Tree

52 44 35 23 20 18 12

Right-node-left traversal of a binary search tree produces a descending sequence

Three BST search algorithms:

- Find the smallest node
- Find the largest node
- Find a requested node

ALGORITHM 7-1 Find Smallest Node in a BST

```
Algorithm findSmallestBST (root)
This algorithm finds the smallest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of smallest node
1  if (left subtree empty)
   1   return (root)
2  end if
3  return findSmallestBST (left subtree)
end findSmallestBST
```

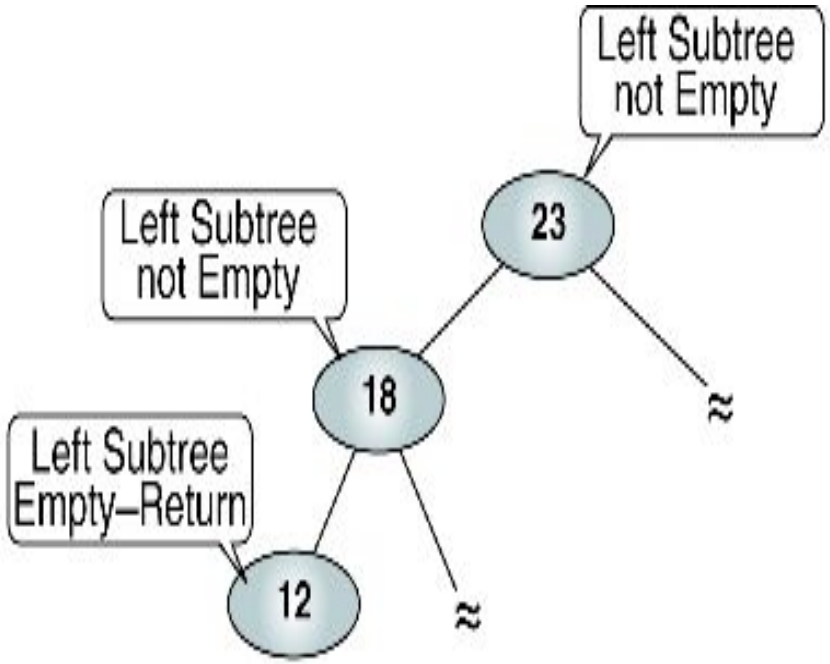


FIGURE 7-5 Find Smallest Node in a BST

ALGORITHM 7-2 Find Largest Node in a BST

```
Algorithm findLargestBST (root)
This algorithm finds the largest node in a BST.
   Pre    root is a pointer to a nonempty BST or subtree
   Return address of largest node returned
1  if (right subtree empty)
   1   return (root)
2  end if
3  return findLargestBST (right subtree)
end findLargestBST
```

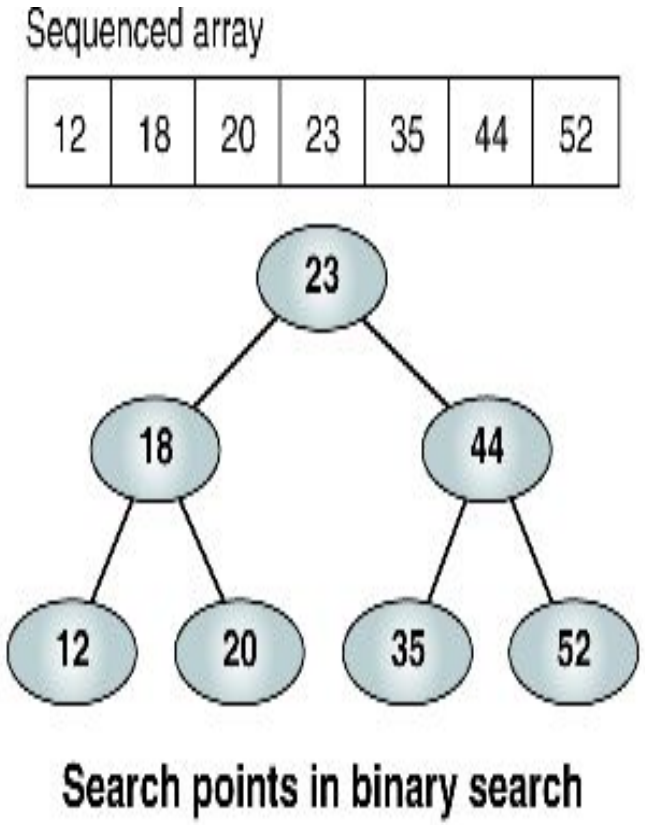


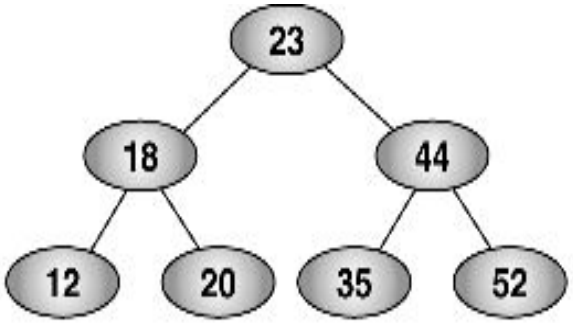
FIGURE 7-6 BST and the Binary Search

ALGORITHM 7-3 Search BST

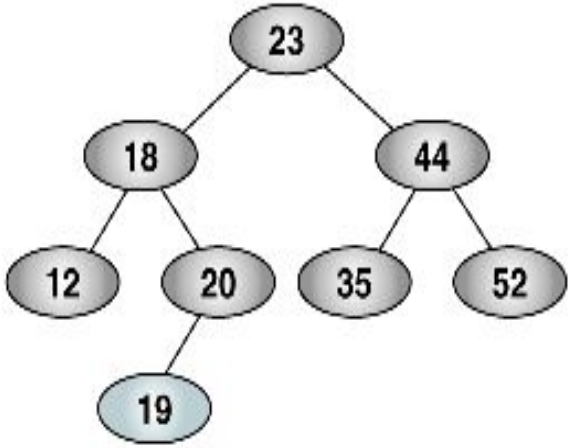
```
Algorithm searchBST (root, targetKey)
Search a binary search tree for a given value.
   Pre    root is the root to a binary tree or subtree
          targetKey is the key value requested
   Return the node address if the value is found
          null if the node is not in the tree
1  if (empty tree)
      Not found
   1  return null
2  end if
3  if (targetKey < root)
   1  return searchBST (left subtree, targetKey)
4  else if (targetKey > root)
   1  return searchBST (right subtree, targetKey)
5  else
      Found target key
   1  return root
6  end if
end searchBST
```

BST Insertion

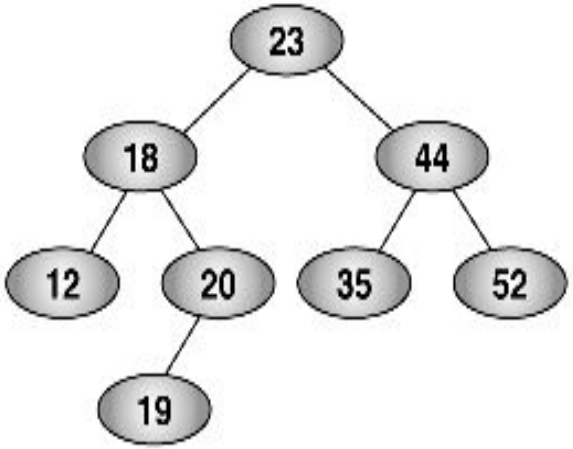
- To insert data all we need to do is follow the branches to an empty subtree and then insert the new node.
- In other words, all inserts take place at a leaf or at a leaflike node – a node that has only one null subtree.



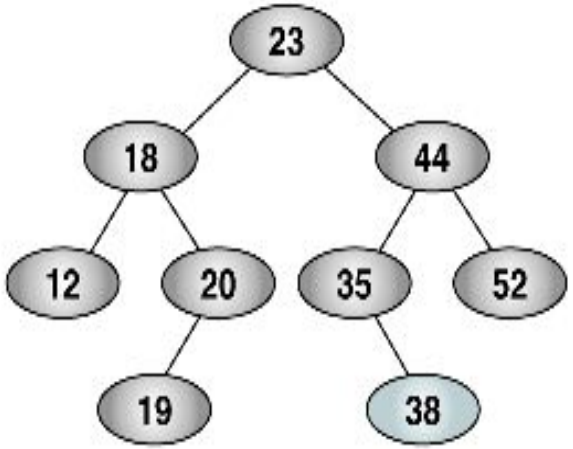
(a) Before inserting 19



(b) After inserting 19



(c) Before inserting 38



(d) After inserting 38

FIGURE 7-8 BST Insertion

ALGORITHM 7-4 Add Node to BST

```
Algorithm addBST (root, newNode)
Insert node containing new data into BST using recursion.
   Pre    root is address of current node in a BST
          newNode is address of node containing data
   Post   newNode inserted into the tree
   Return address of potential new tree root
1  if (empty tree)
   1   set root to newNode
   2   return newNode
2  end if
```

continued

ALGORITHM 7-4 Add Node to BST *(continued)*

```
   Locate null subtree for insertion
3  if (newNode < root)
   1   return addBST (left subtree, newNode)
4  else
   1   return addBST (right subtree, newNode)
5  end if
end addBST
```

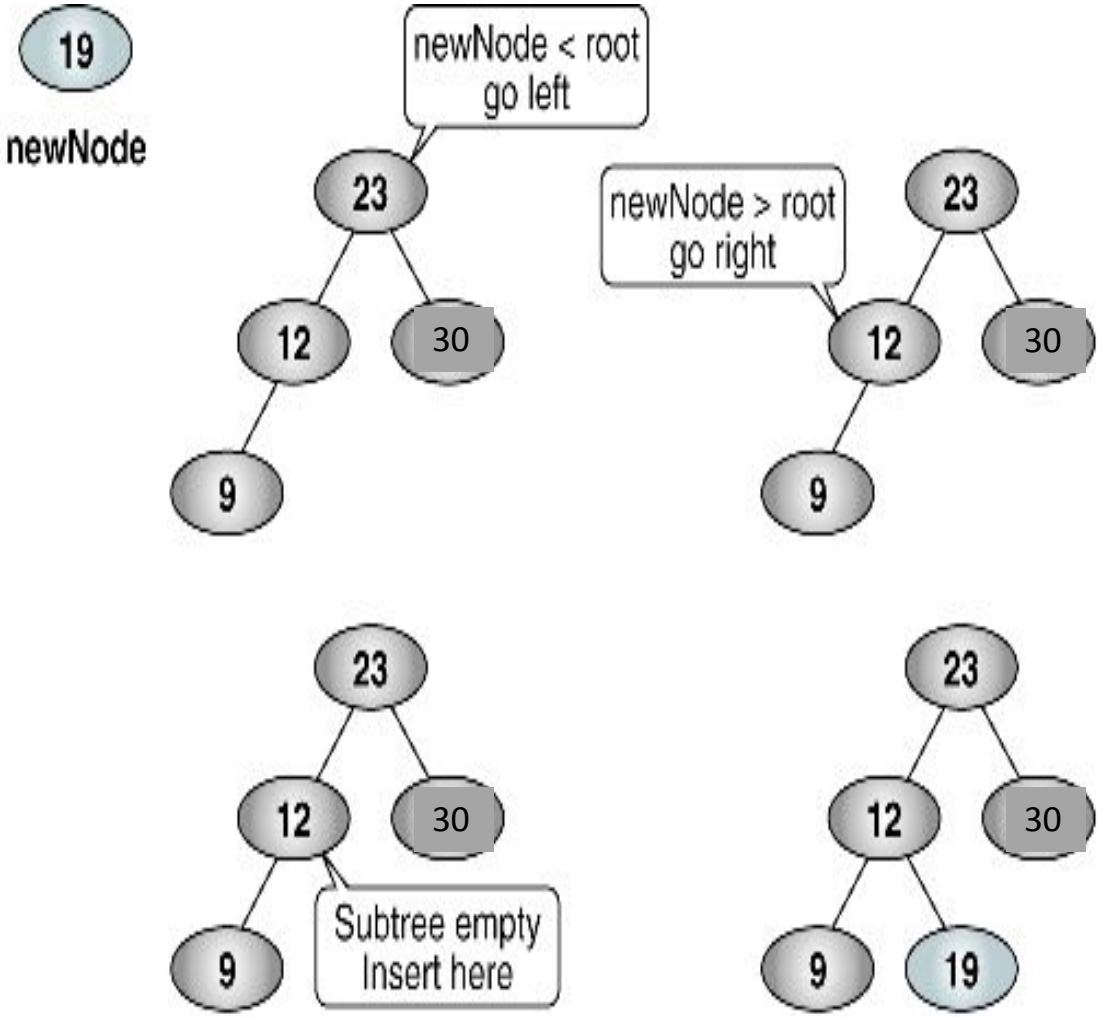


FIGURE 7-9 Trace of Recursive BST Insert

Deletion

- There are the following possible cases when we delete a node:
- The node to be deleted has no children. In this case, all we need to do is delete the node.
- The node to be deleted has only a right subtree. We delete the node and attach the right subtree to the deleted node's parent.
- The node to be deleted has only a left subtree. We delete the node and attach the left subtree to the deleted node's parent.
- The node to be deleted has two subtrees. It is possible to delete a node from the middle of a tree, but the result tends to create very unbalanced trees.

Deletion from the middle of a tree

- Rather than simply delete the node, we try to maintain the existing structure as much as possible by finding data to take the place of the deleted data. This can be done in one of two ways.

Deletion from the middle of a tree

- We can find the largest node in the deleted node's left subtree and move its data to replace the deleted node's data.
- We can find the smallest node on the deleted node's right subtree and move its data to replace the deleted node's data.
- Either of these moves preserves the integrity of the binary search tree.

ALGORITHM 7-5 Delete Node from BST

```
Algorithm deleteBST (root, dltKey)
This algorithm deletes a node from a BST.
   Pre    root is reference to node to be deleted
          dltKey is key of node to be deleted
   Post   node deleted
          if dltKey not found, root unchanged
   Return true if node deleted, false if not found
1  if (empty tree)
   1   return false
2  end if
3  if (dltKey < root)
   1   return deleteBST (left subtree, dltKey)
4  else if (dltKey > root)
   1   return deleteBST (right subtree, dltKey)
5  else
       Delete node found--test for leaf node
   1   If (no left subtree)
       1   make right subtree the root
       2   return true
```

ALGORITHM 7-5 Delete Node from BST (continued)

```
   2  else if (no right subtree)
      1  make left subtree the root
      2  return true
   3  else
         Node to be deleted not a leaf. Find largest node on
         left subtree.
      1  save root in deleteNode
      2  set largest to largestBST (left subtree)
      3  move data in largest to deleteNode
      4  return deleteBST (left subtree of deleteNode,
                           key of largest
   4  end if
 6 end if
end deleteBST
```

Lecture No.33

B-Tree

...in its right sub-tree is longer than left

...known as balanced if the longest path in left and right sub-trees are identical.

8.14 B-TREE (BALANCED MULTI-WAY TREE)

Binary search tree is, in general called multi-way search tree. The integer m is called the order of the tree. Each node should have maximum m children. If $k \leq m$, where m is number of children, then node has accurately k-1 keys which divides all the keys into k number of sets. In case some sets are empty, the children are also empty.

The B-tree is also known as the balanced sort tree. The B-tree is used in external sorting. The B-tree is not a binary tree. While implementing B-tree following conditions are followed:

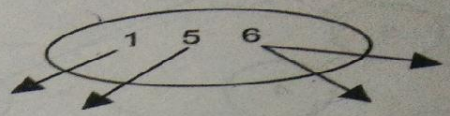
1. The height of the tree must be minimum.
2. There should be no empty sub-trees after the leaves of the tree.
3. The leaves of the tree should be at the same level.
4. All nodes excepting the leaves should have at least few children.

8.14.1 B-Tree Insertion

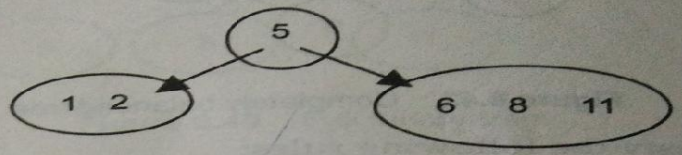
In B-tree insertion at first search is made where the new element is to be placed. If the node is suitable to the given element then insertion is straightforward. The element is inserted by using an appropriate pointer in such an order that number of pointers will be one more than the number of records. In case, the node overflows due to upper bound of node, splitting is mandatory. The node is divided into three parts. The middle part is passed upward. It will be inserted into the parent. Partition may spread the tree. This is because the parent into which element is to be inserted spilts into its child nodes. If the root is needed to be split, a new root is created with two children. The tree grows by one level.

Example: Consider the following B-tree of degree 4. It can be balanced in four ways. Here, each node holds elements. It also has four branches. Suppose, it has the following values:

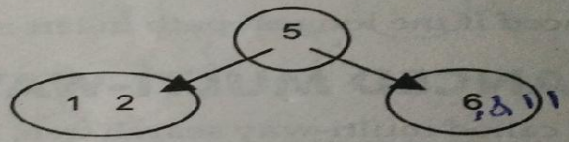
1 5 6 2 8 11 13 18 20 7 9



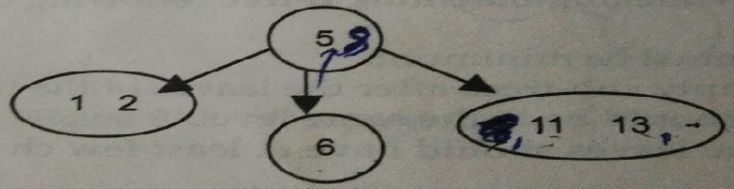
The value 1 is put in a new node. This node can also hold next two values.



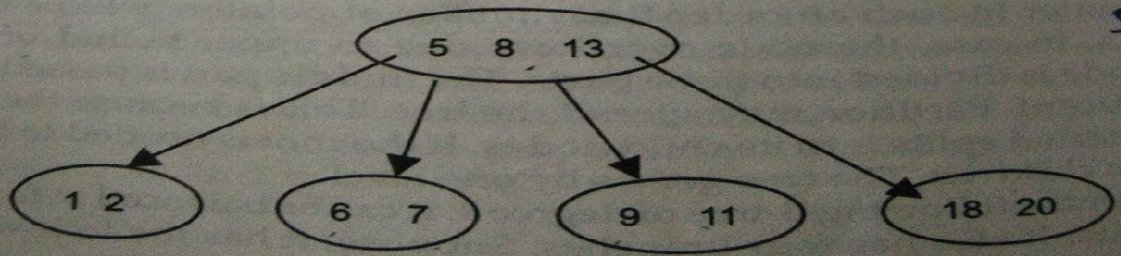
When value 2 (4th value) is put, the node is split at 5 into leaf nodes. Here, 5 is parent. The element 8 is added in leaf node. The search for its accurate position is done in the node having value 6. The element 8 also is present in the same node.



The element 13 is to be inserted. However, the right leaf node, in which 1 to 3 values have appropriate plane, is occupied. Hence, the node splits at median 8 and this moves it up to the parent.



By following the above procedure the remaining nodes can be included. The final figure would be as follows:



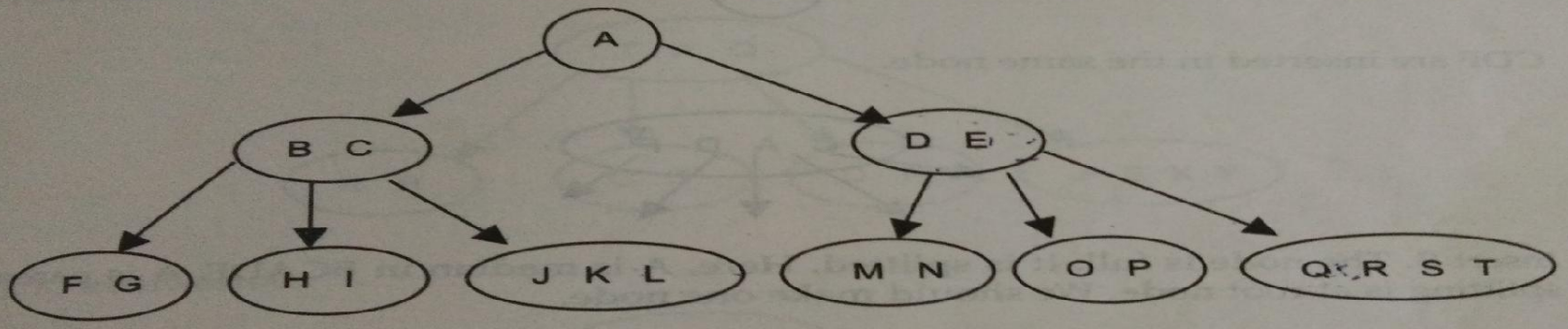
8.14.2 B-Tree Deletion

In B-Tree deletion the element which is to be deleted is searched. In case the element is terminal node, the deletion process is straightforward. The element with a suitable pointer is deleted.

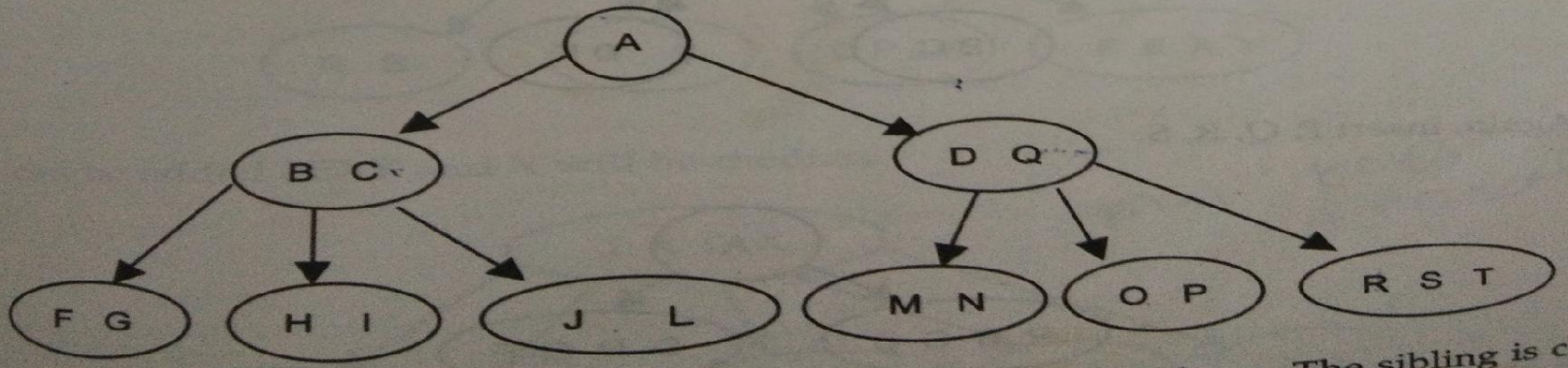
If the element fixed for deletion is not a terminal node, it is replaced by its successor element. Actually, a copy of successor is taken. The successor is an element with higher value. The successor of any node which is not at lowest level, be a terminal node. Thus, deletion is nothing but removing of a particular element or record from a terminal node. While deleting the record the new node size is more than minimum, i.e. the deletion is complete. If the node size is less than minimum, an underflow happens.

Rearrangement is done if either of adjacent siblings has more than the minimum elements (records). For rearrangement, the contents of the node (only those nodes having less than minimum records) along with sorting out records from parent node are gathered. The central record is written back to the parent and left and right halves are written back to two siblings.

Concatenation is applied if the node with less than minimum number of records has no adjacent sibling. The node is combined with its neighbouring sibling and element is moved from its parent.

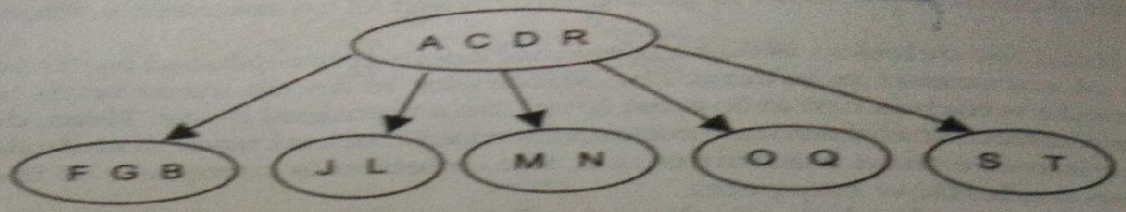


1. Deleting K is straightforward because it is a leaf node.
2. Deleting E is not simple. Hence, its successor is moved up. E is moved down and deleted.



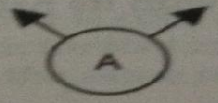
3. To delete P, the node has less than minimum numbers of keys. The sibling is carried. R moves up and Q moves down.

4. Deleting H, again node has less than minimum keys than required. The parent is left with only one key. Here, sibling cannot be applied. Hence, A, C, D and R form a new node.

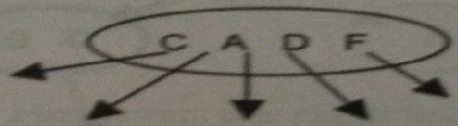


8.15 B-TREE OF ORDER 5

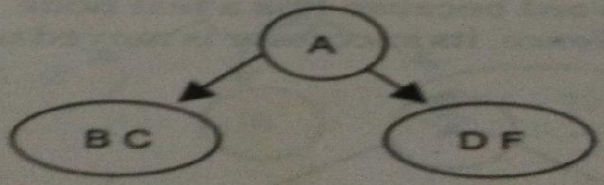
We can build a B-tree of order 5.



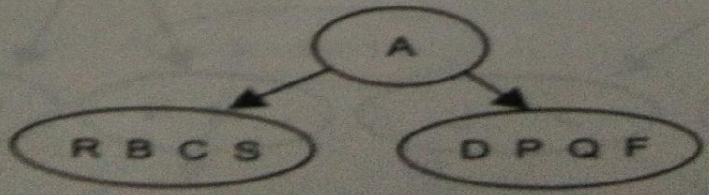
1. CDF are inserted in the same node.



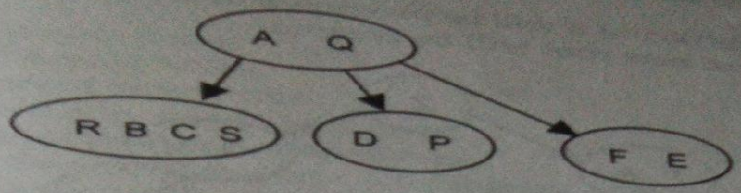
2. Insert B. The node is full. It is splitted. Here, A is median in BCADF. A is parent. The splitting is at root node. We should make one node.



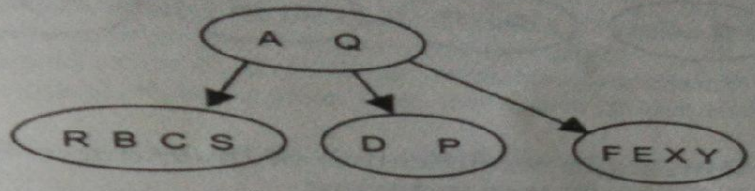
3. Again, insert P, Q, R, S.



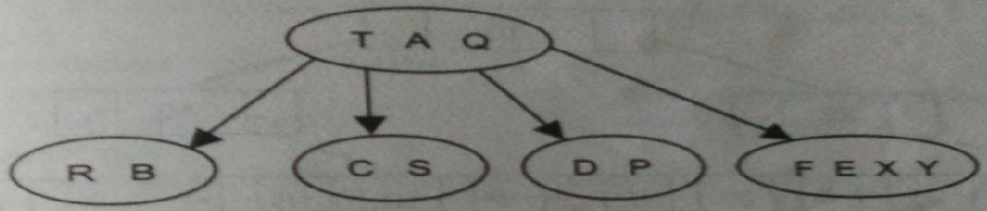
4. E can be added in DPQF and here Q is median.



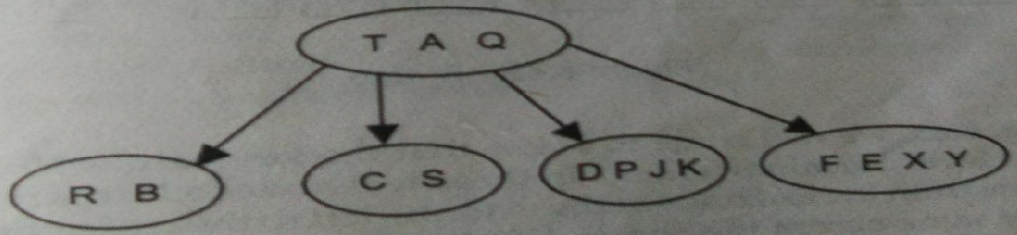
5. X and Y are entered after Q.



6. After addition of T.

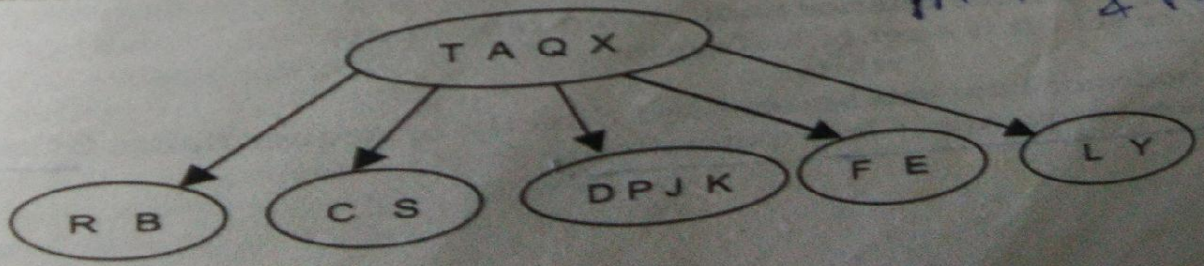


7. J, K are inserted.



8. L can be added FEXY and X will be median.

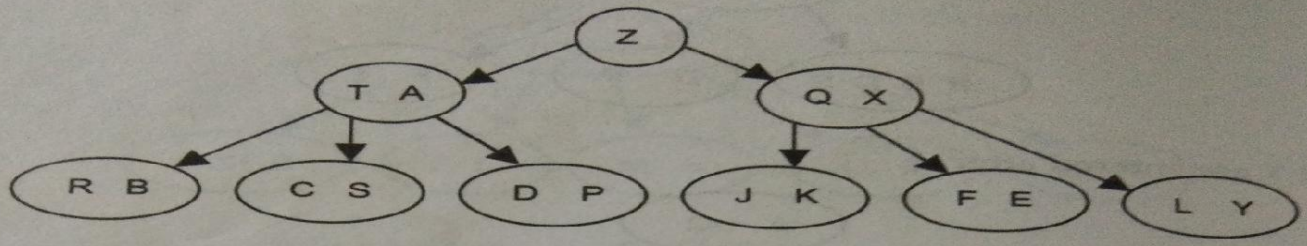
m-1 keys & m leaf nodes



Lecture No.34

B+ Tree

9. Z can be placed in DPJK and it will be promoted to TAQX. But this is also full and hence, the root will split and new root will be created.



8.16 B+ TREE

A B+ tree can be obtained with slight modification of indexing in B tree. A B+ tree stores key values repeatedly. Fig. 8.43 shows B+ tree indexing.

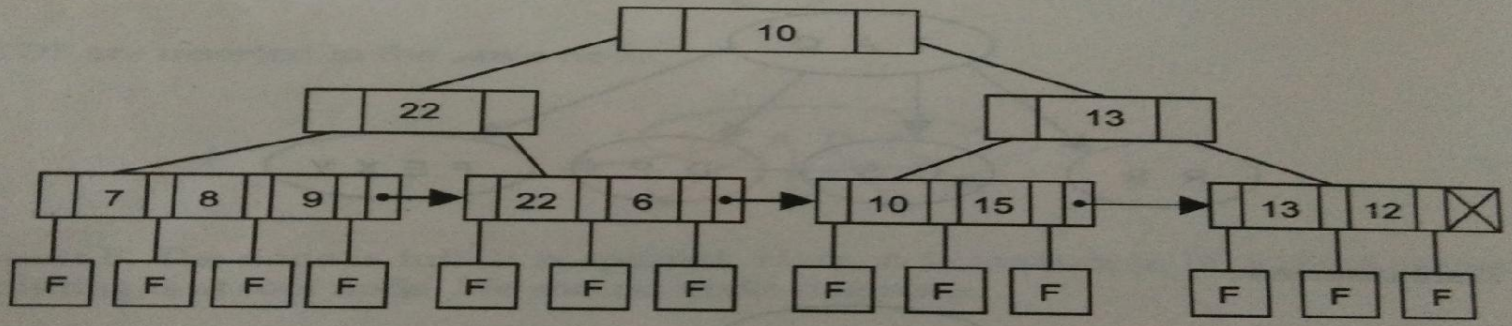


Figure 8.43 B+ tree

By observing Fig. 8.43, we will come to know that the key values 10, 22 and 13 are stored repeatedly in the last level (terminal nodes). In these leaf nodes a link is maintained so that the traversing can be done from the leaf node at the extreme left to the leaf node at the extreme right. B tree and B+ tree are same except the above differences.

Every leaf node of B+ tree has two parts:

1. **Index part** It is the interior node stored repeatedly.
2. **Sequence set** It is a set of leaf nodes.

The interior nodes or keys can be accessed directly or sequentially. B+ tree is useful data structure in indexed sequential file organization.